PROJECT REPORT: PHISHING AWARENESS SIMULATION

> **Project Title:** Phishing Awareness Simulation using Social Engineering
> **Project Category:** Major Project: Cyber Security
> **Author:** Aditya Sonune

ABSTRACT

With the rise of cyber threats, organizations and individuals are frequently targeted by phishing campaigns. This project focuses on understanding how these attacks work in practice by building a realistic phishing simulation from scratch. The simulation mimics a typical Microsoft 365 credential harvesting attack. I developed a Python Flask backend to host the fake login page and track interactions, and deployed it on Vercel with a Firebase Firestore database. By sending out a fake "password expiry" email to a test group, I recorded how many users clicked the link and attempted to log in. The project demonstrates how easily users can be tricked by familiar design and urgency, and highlights why technical defenses must be combined with user education.

1. INTRODUCTION

Phishing is one of the most common ways attackers steal passwords and gain unauthorized access to systems. Instead of hacking into a server, attackers simply trick users into handing over their credentials. For this project, my goal was to set up a controlled phishing simulation to see exactly how effective these attacks are and to understand the infrastructure behind them. The main objective was to build a convincing fake login portal, send out a lure email to a few volunteers, and track their responses securely—without actually saving any of their real passwords.

2. SYSTEM ARCHITECTURE & IMPLEMENTATION

To make the simulation realistic and accessible, I built a custom web application and deployed it to the cloud.

- **Frontend Design:** I cloned the Microsoft 365 login page using HTML and CSS. It was important to match the exact colors, typography, and layout so that users wouldn't get suspicious when the page loaded.

- **Backend Server:** I wrote the backend using Python and the Flask framework. The server has two main jobs: serving the cloned webpage and logging when a user interacts with it. I created a specific route for the form submission that extracts the email address but deliberately drops the password data before logging it.
- **Deployment & Database:** I deployed the Flask application to Vercel so that it would be accessible over the internet with a clean HTTPS connection (`microsoft-account-protection-hub.vercel.app`). For tracking the results, I integrated Firebase Firestore to store interaction data locally and in the cloud.

3. METHODOLOGY

The actual simulation was carried out in three distinct steps:

- **The Lure:** I created an email template that looked like an automated alert from IT Support, sent from `admin@microsoft-security-auth.com`. It informed the user that their Microsoft 365 password would expire in 24 hours. This created a sense of urgency, which is a common social engineering tactic.
- **The Hook:** The email contained a "Update Password" link pointing to my deployed Vercel application.
- **Execution:** I sent the email to 10 participants. If a participant clicked the link, a `PAGE_VISIT` event was logged. If they proceeded to enter their details, a `SUBMIT_CREDENTIALS` event was recorded. Afterward, they were redirected to a disclosure page explaining that this was an educational simulation.

4. RESULTS AND ANALYSIS

After running the simulation, I checked the Firebase database and local logs to compile the results. Out of the 10 emails sent, the interaction numbers were quite revealing:

- **Total Emails Sent: 10**
- **Total Link Clicks: 7 (70% click rate)**
- **Logins Attempted: 3 (30% compromise rate)**
- **Conversion Rate: ~43% of the people who clicked actually tried to log in.**

The data shows that the urgency of the email was very effective at getting people to click. However, once on the page, some users noticed the URL wasn't actually `microsoft.com` and closed it. Still, 3 users trusted the familiar UI and submitted their emails, proving that brand familiarity often overrides caution.

5. SCREENSHOTS

5.1 Phishing Lure (Email Body)

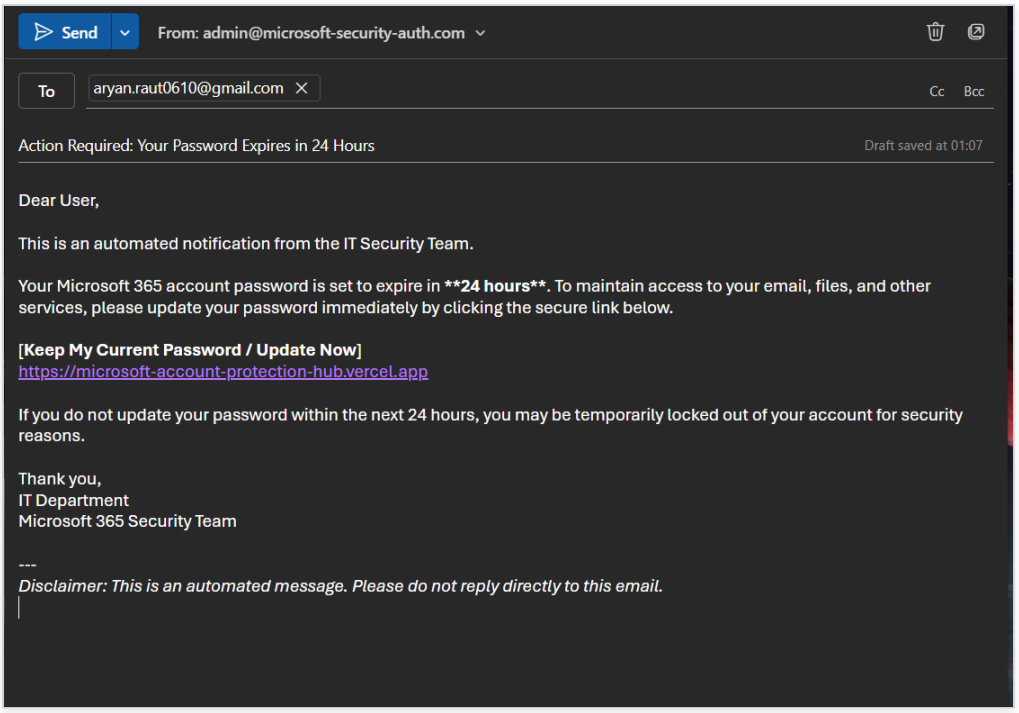

Figure 1: The fake password expiry notification sent to users.

5.2 Simulation Portal (Login UI)

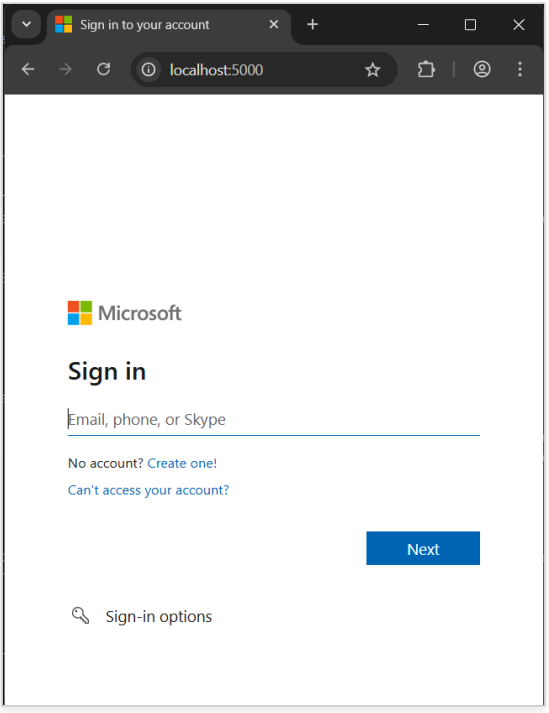

Figure 2: The cloned Microsoft 365 authentication portal.

5.3 Backend Logging (Real-time Tracking)

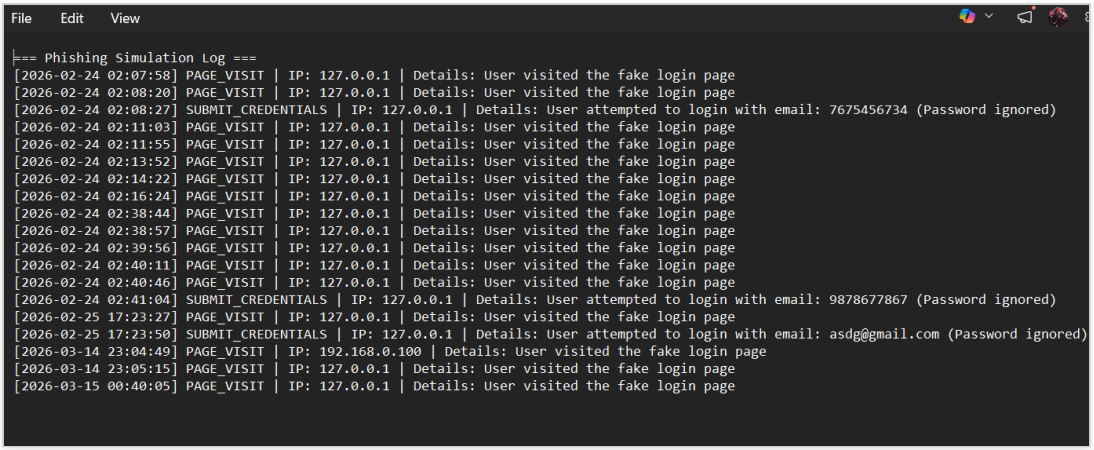

Figure 3: Interaction records recorded in the local text file log.

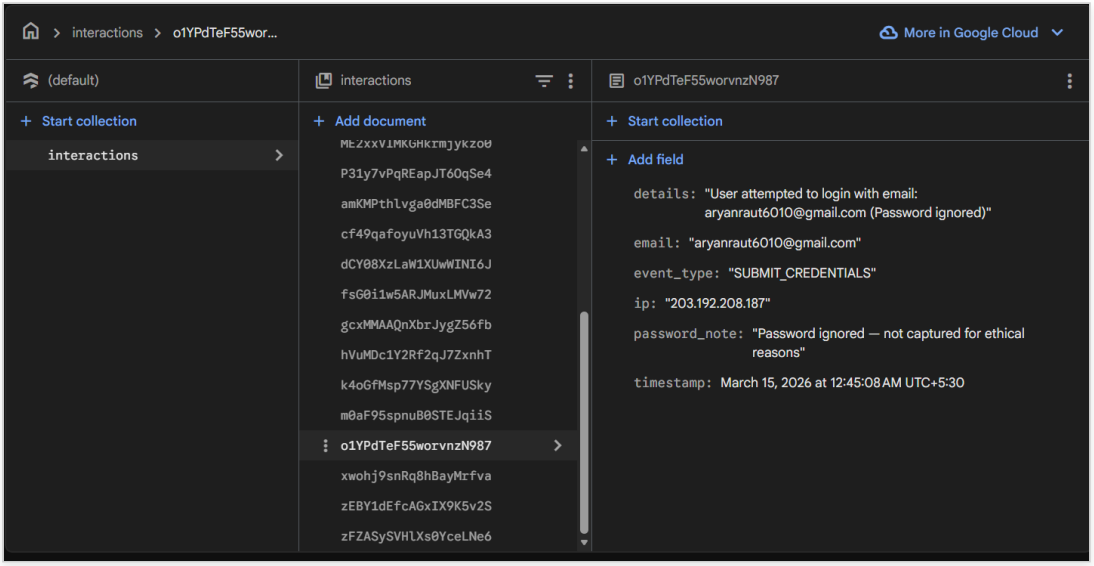

Figure 4: Data permanently stored in the Firebase Firestore cloud database.

5.4 Education & Disclosure Page

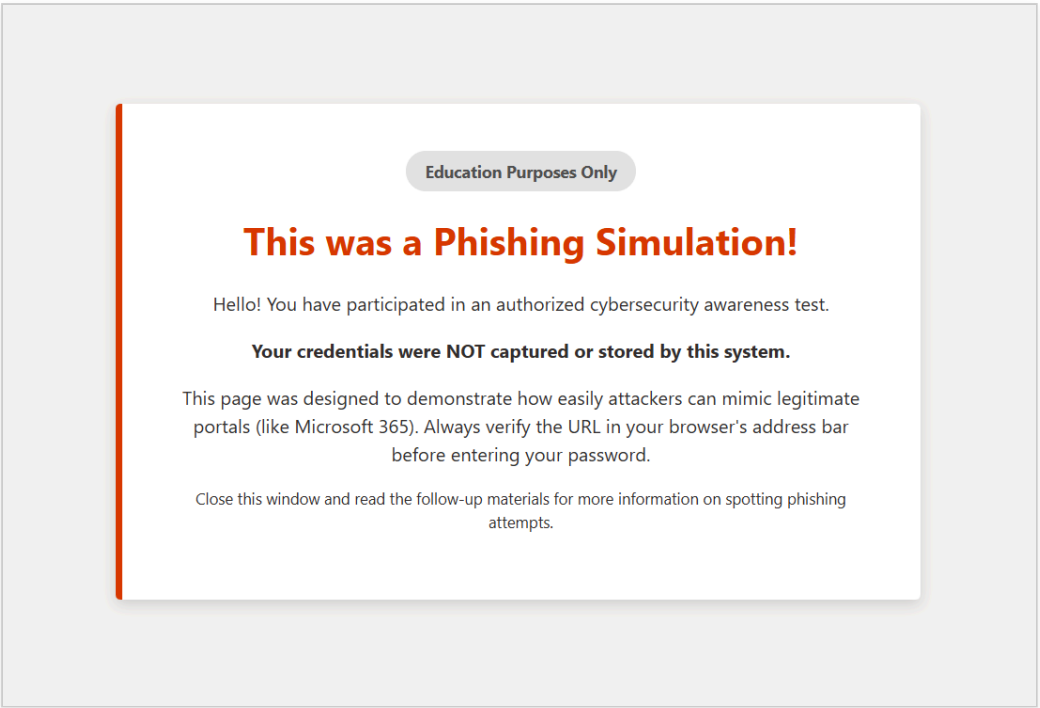

Figure 5: The warning page shown to users after interaction.

6. MITIGATION AND PREVENTIVE MEASURES

From observing this simulation, there are a few practical ways organizations can stop these attacks:

- **Check the Sender:** Users need to look at the actual email address, not just the display name. Attackers often spoof the display name easily.
- **Verify the URL:** Even if a website looks identical to the real one, inspecting the URL in the browser's address bar is the most reliable way to spot a fake site.
- **Use Multi-Factor Authentication (MFA):** If an attacker successfully steals a password, having MFA enabled stops them from logging in, adding a crucial layer of security.
- **Continuous Education:** Running simulations like this one helps practically train users to spot fake emails and suspicious links before they click.

7. CONCLUSION

Building this phishing simulation showed me firsthand how simple it is to set up a credential harvesting site using modern web tools like Flask and Vercel. It also proved that human error remains one of the biggest vulnerabilities in cybersecurity. The project successfully demonstrated the attack chain and provided valuable insights into user behavior when faced with urgency and familiar branding. The simulation served as a strong reminder that technical security measures must be paired with ongoing security awareness training.